DRAFT — PENDING OWNER APPROVAL

AutoClip Access Control Policy — Candidate

Document ID: AUTOCLIP-ACP-CANDIDATE **Version:** Draft 1.1 **Status:** DRAFT — PENDING OWNER APPROVAL **Accountable business operator:** 56.151.919 DIOGO NASCIMENTO DOS SANTOS, MEI holder, CNPJ 56.151.919/0001-30 **Contact:** diogos.nascimento@outlook.com (monitoring not verified) **Effective date / approval:** PENDING OWNER DECISION **Publication status:** Not published. This candidate does not replace the public security-policy HTML or prove an existing access-control policy.

1. Purpose

AutoClip is the trade name of the MEI business identified above. The legal operator is accountable for this candidate; controller/processor roles are determined per processing activity.

Define requirements for granting, using, reviewing and revoking access to AutoClip information and systems, including personal information, workspace content, OAuth credentials, production environments and administrative consoles.

2. Scope

Applies to the application, PostgreSQL/Supabase Auth and Storage, Edge Functions, workers, source code, hosting/provider consoles, support processes, personnel and third parties with access. Actual deployed configuration must be verified separately; local migrations and tests do not establish production enforcement.

3. Responsibilities

- The project owner approves the policy, defines the authorized administrator roster and assigns access reviewers.
- System owners grant access only after an approved request and record its purpose, scope and expiry.
- Workspace owners/admins manage only their workspace and may not grant platform-level privileges.
- Users and providers protect credentials and promptly report suspected compromise.
- A reviewer independent of the request where practicable checks privileged access and exceptions.

Named personnel, delegate coverage and approval are pending owner confirmation.

4. Least privilege and role-based access

Access must be individually attributable, limited to the minimum required function and duration, and separated between ordinary user and administrative duties. Shared accounts should not be used unless a documented service identity is necessary and individually controlled. Privileged access must not be granted through client-controlled data or an untrusted request parameter.

5. Administrative access

Production and provider-console access must use named accounts, MFA where supported, restricted membership, protected recovery methods and periodic review. Service credentials must remain server-side, be scoped to required operations, be protected from source control and browser delivery, and be rotated/revoked under a documented process. Current MFA, account roster, secret-store scope and production settings are not proven by this candidate.

6. Workspace authorization and roles

Each tenant operation must verify the authenticated identity, active workspace membership, role and target-object relationship. Owner/Admin/Editor/Viewer permissions must be enforced consistently by database RLS, RPCs, Edge Functions, workers and Storage policies. Cross-workspace references and role changes must be denied unless a separately authorized platform operation is documented. Service-role execution must not be exposed to ordinary users.

Local migrations and tests contain workspace-scoped access controls. Their presence does not prove the Cloud policies or complete runtime isolation. RLS metadata alone is not a substitute for negative cross-tenant tests in an approved isolated environment.

7. Credential protection

Credentials, OAuth tokens, recovery codes, private keys and secrets must not be shared in chat, committed to the repository, included in client responses, or written to ordinary logs. Local token encryption code and tests provide local evidence only. Deployed key provisioning, access, rotation and telemetry redaction require separate verification.

8. Access requests and provisioning

Every grant/change must identify the requester, approver, user/service identity, role, workspace/system scope, business purpose, start date and expiry or review date. Verify identity before creating access. Apply least privilege and test the intended path without using another tenant's data. Privileged grants require owner or delegated approval.

9. Access review and revocation

Review privileged access at least quarterly and workspace membership on role change, separation, user request or suspected compromise. Remove access promptly when the purpose ends. Revoke active sessions, credentials and OAuth authorizations as appropriate, while documenting residual provider/backup retention. Record reviewer, date, findings and closure evidence.

10. Emergency access

Emergency access must be time-limited, justified, approved where practicable, logged and reviewed after use. Preserve evidence and avoid using emergency access to bypass workspace authorization or change production without the required change approval.

11. Records and monitoring

Maintain an access request/change/review record that contains no secret values. Monitor and review authentication anomalies, denied access and privileged changes according to verified system capabilities. Cloud logging, retention, alert rules and access to logs remain to be confirmed.

12. Exceptions

Document the control, reason, scope, affected data, risk, compensating safeguards, accountable approver, expiry, review date and remediation. High-risk exceptions must block the affected access until approval.

13. Approval and version history

Version	Date	Change	Approver	Status
Draft 1	2026-11-02	Initial candidate based on local source and evidence review	Project documentation review	Pending
Draft 1.1	2026-11-02	Added owner-provided MEI identity and privacy/security contact; preserved pending status	Local document update	Pending owner approval

Approval decision: [PENDING]. Effective date: [PENDING]. Publication location: [PENDING]. No approval or publication is implied.